only place that two locks are acquired is in `pop_head()`, which always acquires the `head_mutex` and then the `tail_mutex`, so this will never deadlock.

The remaining question concerns the actual possibilities for concurrency. This data structure actually has considerably more scope for concurrency than that from listing 6.2, because the locks are more fine-grained and *more is done outside the locks*. For example, in `push()`, the new node and new data item are allocated with no locks held. This means that multiple threads can be allocating new nodes and data items concurrently without a problem. Only one thread can add its new node to the list at a time, but the code to do so is only a few simple pointer assignments, so the lock isn't held for much time at all compared to the `std::queue<>`-based implementation where the lock is held around all the memory allocation operations internal to the `std::queue<>`.

Also, `try_pop()` holds the `tail_mutex` for only a short time, to protect a read from `tail`. Consequently, almost the entirety of a call to `try_pop()` can occur concurrently with a call to `push()`. Also, the operations performed while holding the `head_mutex` are also quite minimal; the expensive `delete` (in the destructor of the node pointer) is outside the lock. This will increase the number of calls to `try_pop()` that can happen concurrently; only one thread can call `pop_head()` at a time, but multiple threads can then delete their old nodes and return the data safely.

Waiting For An Item To Pop

OK, so listing 6.6 provides a thread-safe queue with fine-grained locking, but it supports only `try_pop()` (and only one overload at that). What about the handy `wait_and_pop()` functions back in listing 6.2? Can you implement an identical interface with your fine-grained locking?

Of course, the answer is, yes, but the real question is, how? Modifying `push()` is easy: just add the `data_cond.notify_one()` call at the end of the function, just like in listing 6.2. Actually, it's not quite that